ISCD Worksheet 3

Anurag Bhunia(bmat2508@isibang.ac.in)

```r
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.1     v readr     2.1.6
## v ggplot2   4.0.1     v stringr   1.6.0
## v lubridate 1.9.4     v tibble    3.3.1
## v purrr     1.2.0     v tidyr     1.3.2

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(viridisLite)
```

Question 1

a)**read.csv**: This is a built-in function in R that reads a CSV file and generates a data frame from it, with cases corresponding to lines and variables to fields in the file.

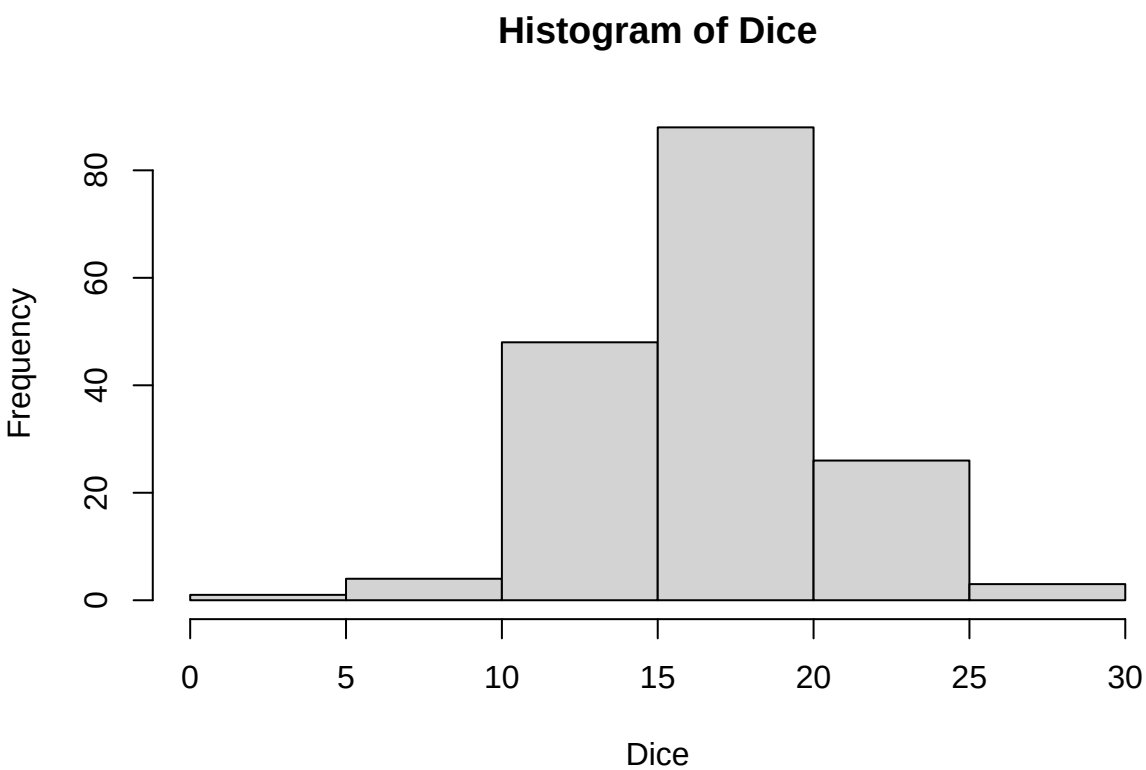
b)**setwd**: R can only access files that are in its current working directory. setwd is used to set the working directory from where reading and writing of files will be done. The syntax is of the form setwd(dir) where dir is a string representing the path of the required directory.

c)**getwd**: getwd returns the absolute filepath of the current working directory.

2.

a)

```r
setwd("/home/anurag/Documents/Dice")
DiceR=read.csv("Dice.csv",header=T)
names(DiceR)="Sum"
DiceR$Sum=as.numeric(DiceR$Sum)
Dice=na.omit(DiceR$Sum)
hist(Dice)
```




```
summary(Dice)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00   15.00   17.00   17.14   20.00   27.00
```

The first line is to create a dataframe from the CSV file Dice.csv using the read.csv function we discussed above. Also, header=T makes it so that R interprets the first row of the CSV file as a list of variable names. The actual data gets considered from the second row onwards.

names(DiceR) is used to get the list of column names from the given dataframe. The CSV file Dice only has one column, so setting names(DiceR) to Sum renames that column to Sum.

DiceR$Sumisusedtoaccessthecolumncalled Suminthisdataframe.as.numeric(DiceR$Sum) takes the whole column vector, converts each entry to a numeric data type and then reassigns that numeric vector to the column.

na.omit(DiceR$Sum) removes all unavailable or conflicting entries from the column vector and setting it to Dice assigns the corrected entries to this new variable.

hist(Dice) takes the entire range of entries in this vector, divides them into bins of appropriate length and then displays the frequency of dice rolls summing to values in that bin.

Finally, summary(Dice) shows a descriptive statistical report of the dataframe by showing markers such as the mean sum, the maximum and minimum sums and so on.

b)

```
setwd("/home/anurag/Documents/Dice")
DiceR=read.csv("Dice.csv",header=T)
names(DiceR)="Sum"
DiceR$Sum=as.numeric(DiceR$Sum)
Dice=na.omit(DiceR$Sum)
cs=(Dice-mean(Dice))/(sd(Dice))
mean(cs)
```

```
## [1] 1.34184e-16
```

```r
sd(cs)
```

```
## [1] 1
```

```r
onesdcs=cs[cs>-1& cs<1]
twosdcs=cs[cs>-2& cs<2]
threesdcs=cs[cs>-3& cs<3]
```

The cs=(Dice-mean(Dice))/sd(DiceR$Sum) is used to standardise the data so that this vector of random variables can be compared with the normal distribution.

The mean of the standardised data is very close to 0, as it should be, since ideally the limitting distribution of the standardised data should follow Normal(0,1).

Similarly, the standard deviation of cs checks out to be 1, as expected.

Finally, the last three lines are recording entries from the vector cs that are atmost one,two or three standard deviations away from the normal on either end.

While the standardisation seems to have worked, we would also like to look at what proportion of the sample lies within onesdcs,twosdcs and threesdcs

```r
setwd("/home/anurag/Documents/Dice")
DiceR=read.csv("Dice.csv",header=T)
names(DiceR)="Sum"
DiceR$Sum=as.numeric(DiceR$Sum)
Dice=na.omit(DiceR$Sum)
cs=(Dice-mean(Dice))/(sd(Dice))
onesdcs=cs[cs>-1& cs<1]
twosdcs=cs[cs>-2& cs<2]
threesdcs=cs[cs>-3& cs<3]
p1=length(onesdcs)/length(cs)
p2=length(twosdcs)/length(cs)
p3=length(threesdcs)/length(cs)
p1
```

```
## [1] 0.6647059
```

```r
p2
```

```
## [1] 0.9588235
```

```r
p3
```

```
## [1] 0.9941176
```

As seen mathematically, for a standard normal variable, p1,p2,p3 should ideally be around 0.68,0.95,0.997 respectively. Since the values here are extremely close to the the ideal values, we can conclude that the normal distribution provides a well enough approximation for the sum of 5 dice rolls.

3.

a) This dataset contains record of daily timeseries of covid19 infections across all states of India as well as a cumulative record of nation-wide infections. There are further records that zero down to district level timeseries during the pandemic. Finally, this dataset also contains record of cowin vaccinations at the district level.

```r
setwd("/home/anurag/Documents/States")
statedf=read.csv("states.csv",header=T)
```

b) The state suiting this requirement as per my name is Haryana.

c)

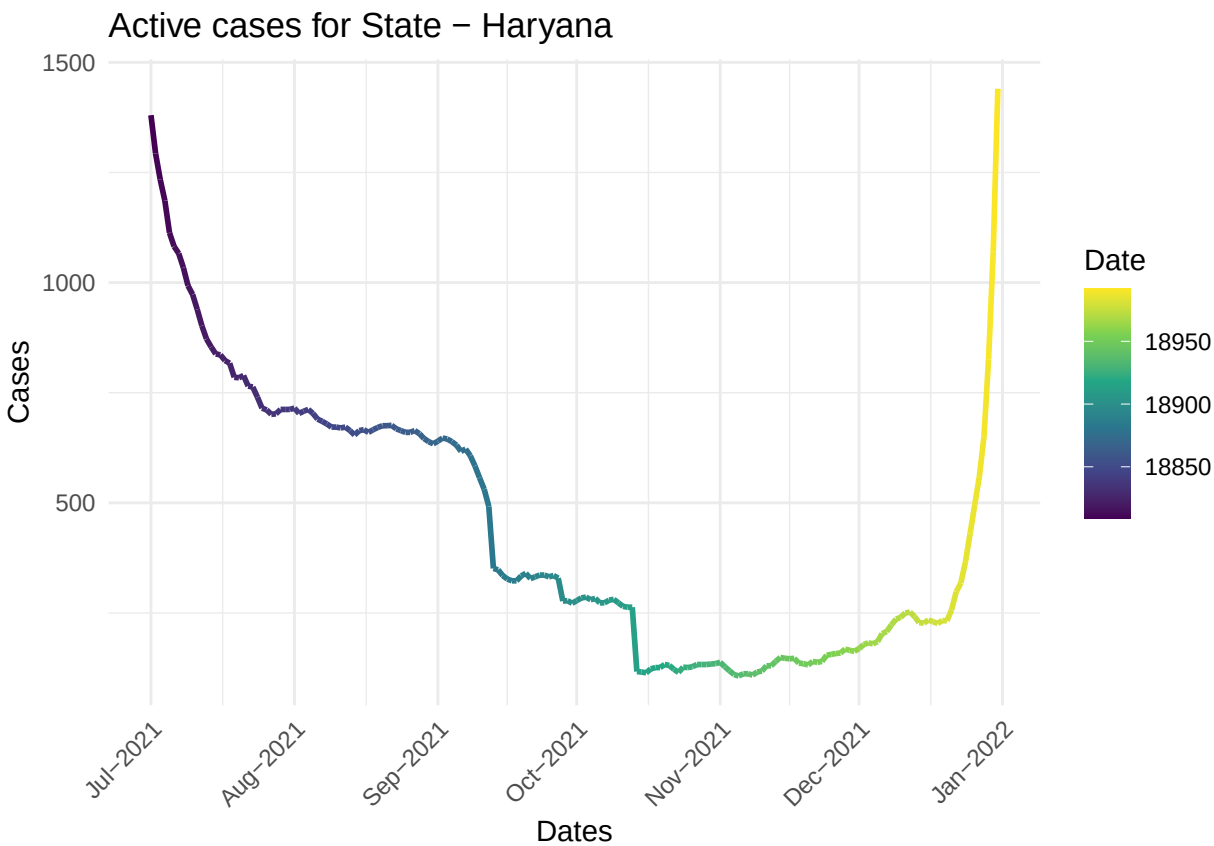
```
setwd("/home/anurag/Documents/States")
statedf=read.csv("states.csv",header=T)
mystatedf=subset(statedf,State=="Haryana")
head(mystatedf)
```

```
##          Date   State Confirmed Recovered Deceased Other Tested
## 19 2020-03-04 Haryana        14         0        0     0      0
## 26 2020-03-05 Haryana        14         0        0     0      0
## 33 2020-03-06 Haryana        14         0        0     0      0
## 40 2020-03-07 Haryana        14         0        0     0      0
## 49 2020-03-08 Haryana        14         0        0     0      0
## 58 2020-03-09 Haryana        14         0        0     0      0
```

d)

```
setwd("/home/anurag/Documents/States")
statedf = read.csv("states.csv", header=TRUE)

# Subset for Haryana
mystatedf = subset(statedf, State=="Haryana")

# Convert Date column
mystatedf$Date = as.Date(mystatedf$Date, format="%Y-%m-%d")

# Compute Active cases
mystatedf$Active = mystatedf$Confirmed - (mystatedf$Recovered + mystatedf$Deceased)

# Filter July 1, 2021 to Dec 31, 2021
plotdf = mystatedf %>% filter(Date>=as.Date("2021-07-01") & Date<=as.Date("2021-12-31"))

# Sort by Date
plotdf = plotdf[order(plotdf$Date),]

# Plot
ggplot(plotdf, aes(x=Date, y=Active, color=Date))+
  geom_line(size=1)+
  scale_color_viridis_c(option="viridis")+
  labs(title="Active cases for State - Haryana", x="Dates", y="Cases", color="Date")+
  scale_x_date(date_breaks="1 month", date_labels="%b-%Y")+
  theme_minimal()+
  theme(axis.text.x=element_text(angle=45,hjust=1))
```

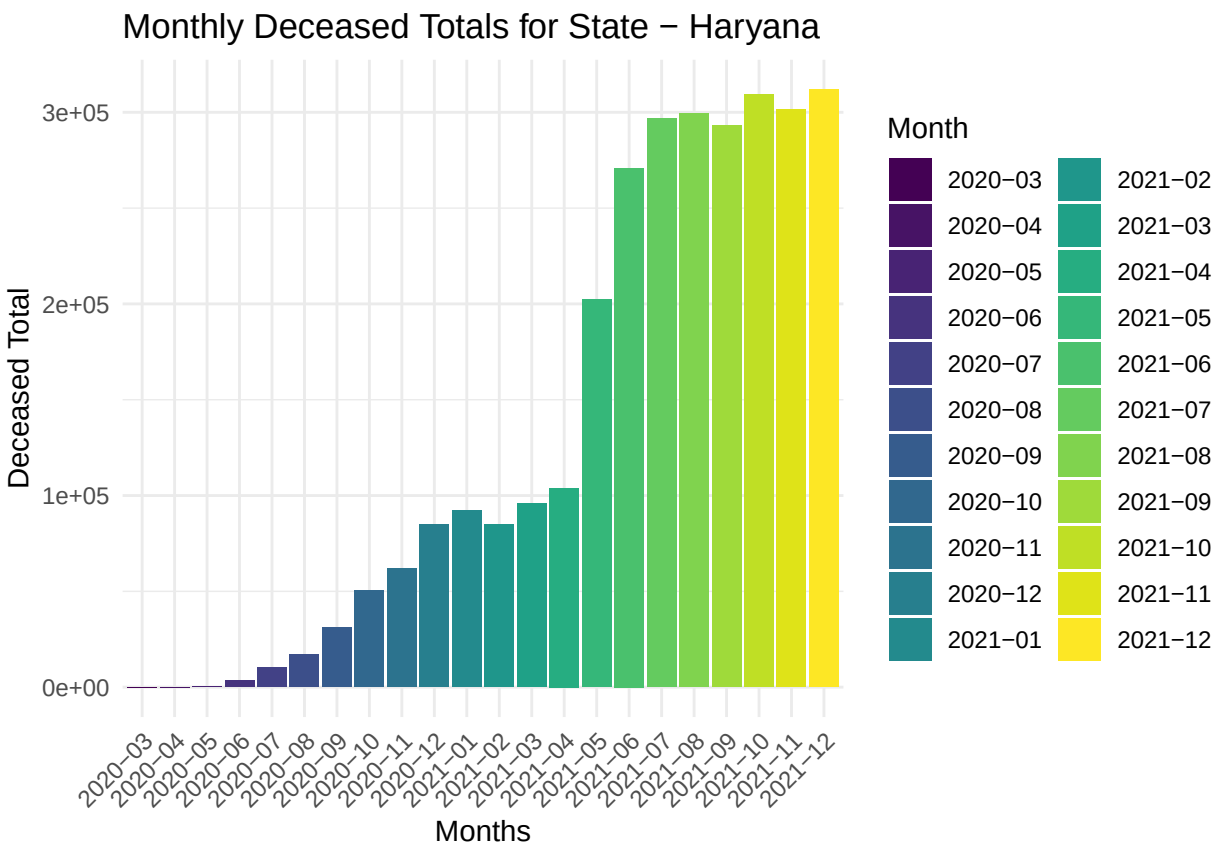
```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



e)

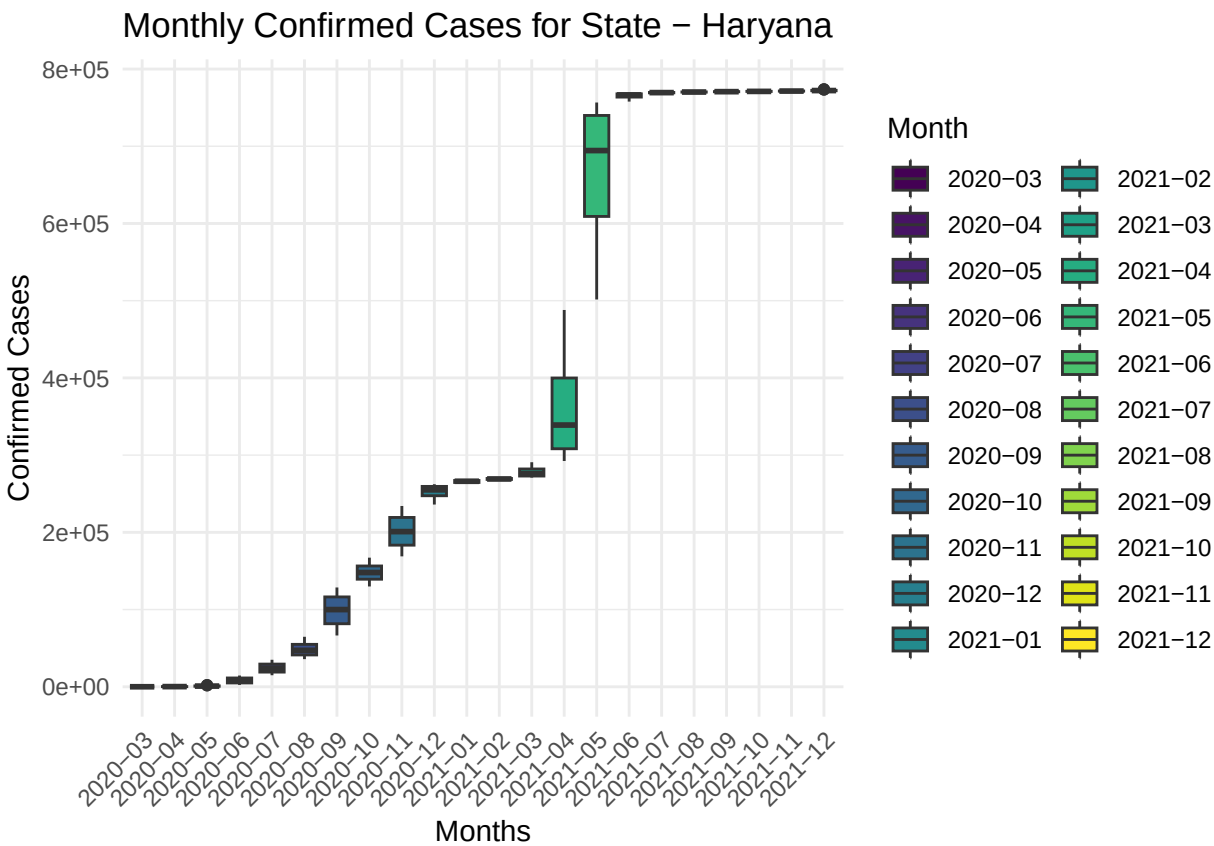
```
setwd("/home/anurag/Documents/States")
statedf = read.csv("states.csv", header=TRUE)
mystatedf = subset(statedf, State=="Haryana")

# Ensure Date column is Date type
mystatedf$Date <- as.Date(mystatedf$Date, format="%Y-%m-%d")

# Create Month column in "YYYY-MM" format
mystatedf$Month <- format(mystatedf$Date, "%Y-%m")

# Filter March 2020 to Dec 2021
mystatedf_filtered <- mystatedf %>%
  filter(Date >= as.Date("2020-03-01") & Date <= as.Date("2021-12-31"))

# Compute total Deceased per month
monthly_deceased <- mystatedf_filtered %>%
  group_by(Month) %>%
  summarise(TotalDeceased = sum(Deceased, na.rm = TRUE)) %>%
  arrange(Month)

# Plot bar chart
ggplot(monthly_deceased, aes(x=Month, y=TotalDeceased, fill=Month)) +
  geom_bar(stat="identity") +
  scale_fill_viridis_d(option="viridis") +
  labs(
    title="Monthly Deceased Totals for State - Haryana",  # replace Haryana if needed
```

```
    x="Months",
    y="Deceased Total",
    fill="Month"
  ) +
  theme_minimal() +
  theme(axis.text.x=element_text(angle=45, hjust=1))
```



f)

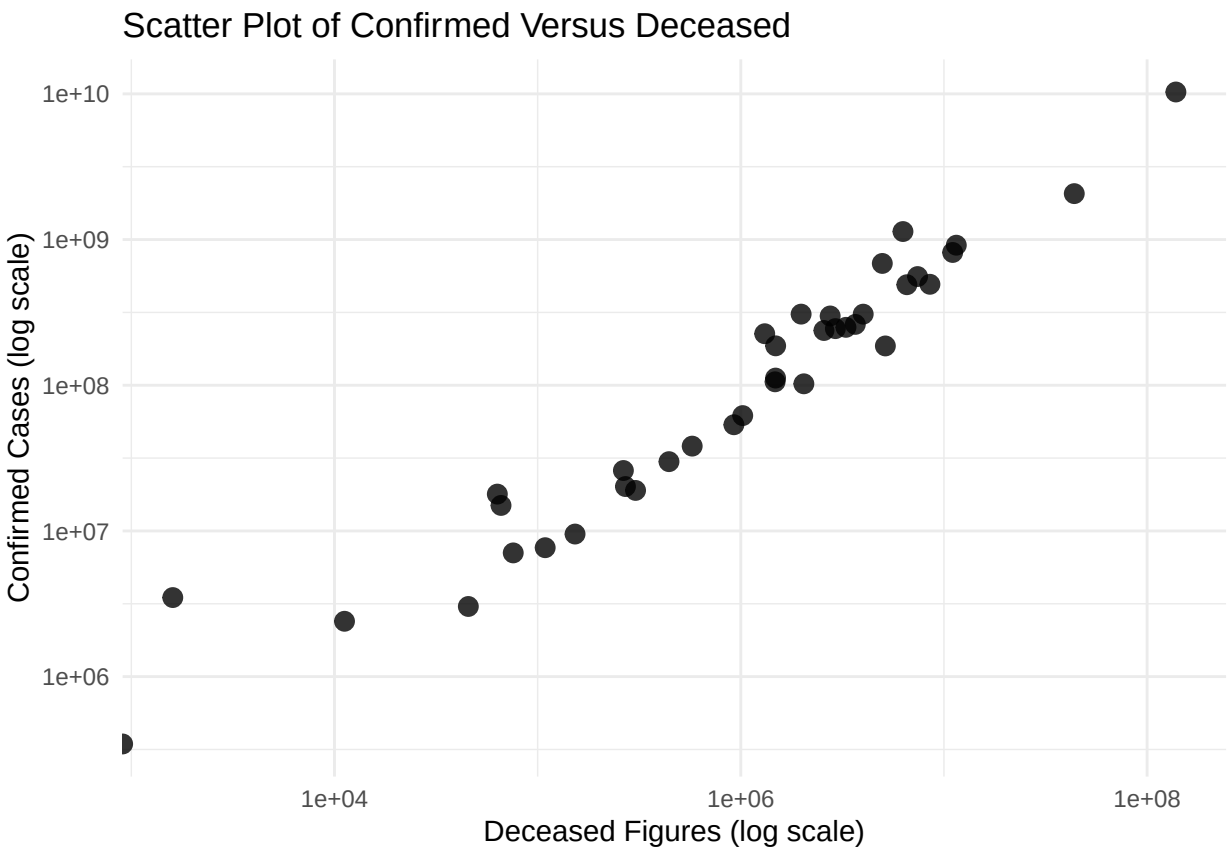
```
setwd("/home/anurag/Documents/States")
statedf = read.csv("states.csv", header=TRUE)
mystatedf = subset(statedf, State=="Haryana")
mystatedf$Date <- as.Date(mystatedf$Date, format="%Y-%m-%d")
mystatedf$Month <- format(mystatedf$Date, "%Y-%m")
mystatedf_filtered <- mystatedf %>%
  filter(Date >= as.Date("2020-03-01") & Date <= as.Date("2021-12-31"))
ggplot(mystatedf_filtered, aes(x=Month, y=Confirmed, fill=Month)) +
  geom_boxplot() +
  scale_fill_viridis_d(option="viridis") +
  labs(
    title="Monthly Confirmed Cases for State - Haryana",
    x="Months",
    y="Confirmed Cases",
    fill="Month"
  ) +
  theme_minimal() +
  theme(axis.text.x=element_text(angle=45, hjust=1))
```



g)

```
setwd("/home/anurag/Documents/States")
statedf = read.csv("states.csv", header=TRUE)
statedf$Date <- as.Date(statedf$Date, format="%Y-%m-%d")
state_totals <- statedf %>%
  filter(Date >= as.Date("2020-03-01"),
         Date <= as.Date("2021-12-31")) %>%
  group_by(State) %>%
  summarise(
    TotalConfirmed = sum(Confirmed, na.rm=TRUE),
    TotalDeceased = sum(Deceased, na.rm=TRUE)
  )
ggplot(state_totals, aes(x=TotalDeceased, y=TotalConfirmed)) +
  geom_point(size=3, alpha=0.8) +
  scale_x_log10() +
  scale_y_log10() +
  labs(
    title="Scatter Plot of Confirmed Versus Deceased",
    x="Deceased Figures (log scale)",
    y="Confirmed Cases (log scale)"
  ) +
  theme_minimal()
```

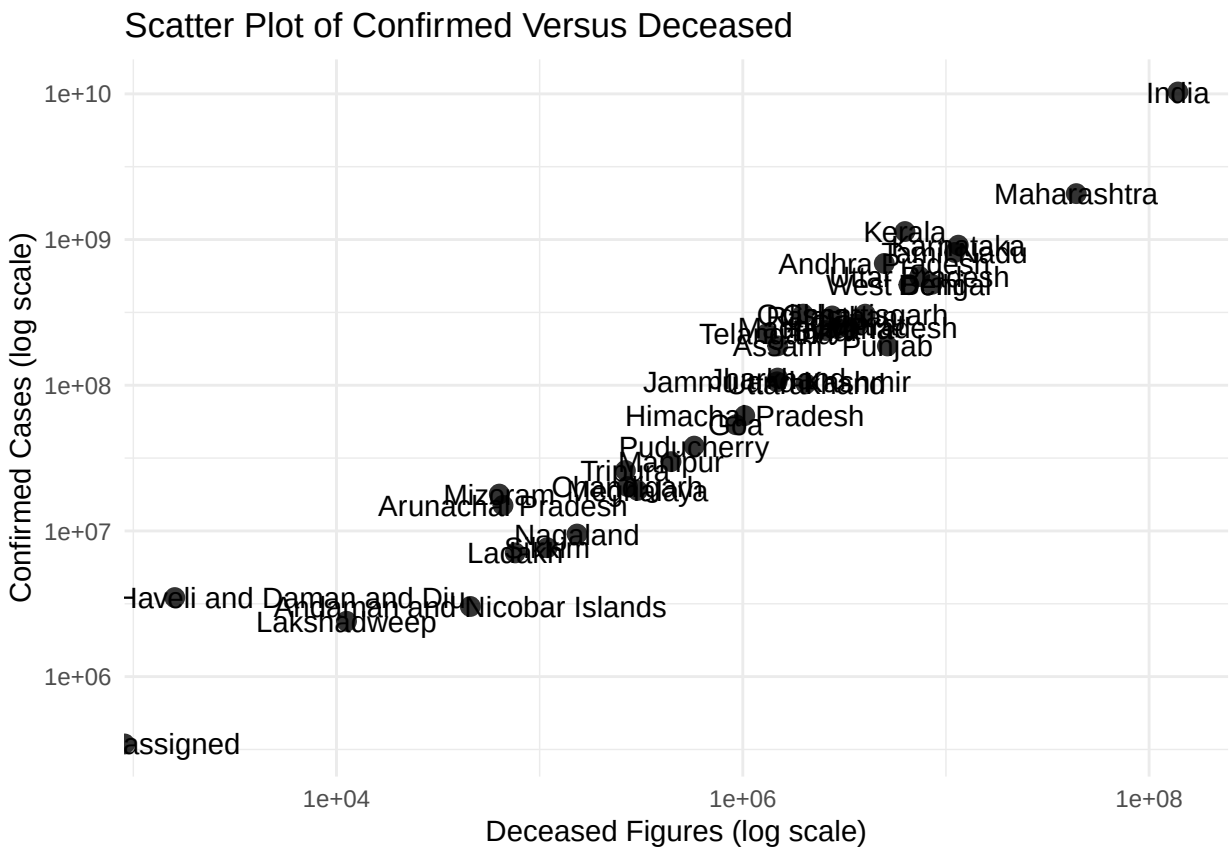
```
## Warning in scale_x_log10(): log-10 transformation introduced infinite values.
```



Technically we can label dots by state names but with so many states, there are likely to be many overlaps; compromising legibility.

```
setwd("/home/anurag/Documents/States")
statedf = read.csv("states.csv", header=TRUE)
statedf$Date <- as.Date(statedf$Date, format="%Y-%m-%d")
state_totals <- statedf %>%
  filter(Date >= as.Date("2020-03-01"),
         Date <= as.Date("2021-12-31")) %>%
  group_by(State) %>%
  summarise(
    TotalConfirmed = sum(Confirmed, na.rm=TRUE),
    TotalDeceased = sum(Deceased, na.rm=TRUE)
  )
ggplot(state_totals, aes(x=TotalDeceased, y=TotalConfirmed)) +
  geom_point(size=3, alpha=0.8) +
  scale_x_log10() +
  scale_y_log10() + geom_text(aes(label=State))+
  labs(
    title="Scatter Plot of Confirmed Versus Deceased",
    x="Deceased Figures (log scale)",
    y="Confirmed Cases (log scale)"
  ) +
  theme_minimal()
```

```
## Warning in scale_x_log10(): log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
```



The above plot shows why trying to label dots by state names might not be such a good idea.